

# ROS 2 二维机器人 运动规划实践

从仿真到定位、规划与控制

本书随代码逐章成长。每个例子都有明确的运行入口，  
每个公式都能在对应函数和实验中找到用途。

**阅读方法** 先运行、看见现象，再读公式和关键代码，最后完成一个小练习。

**版本范围** 当前实际实现范围以仓库进度表为准；本书只收录已实现内容。

主机 ROS 2 Lyrical / RViz2

独立算法 C++17, ROS 节点 C++20

2026 年 9 月

# 目录

|                           |           |
|---------------------------|-----------|
| <b>第一章 环境搭建与第一个圆盘场景</b>   | <b>1</b>  |
| 1.1 先认识四个名字               | 1         |
| 1.2 确认主机环境                | 1         |
| 1.3 构建并启动                 | 2         |
| 1.4 圆盘为什么用直径?             | 3         |
| 1.5 发布一次, 为什么晚启动也能看到?     | 3         |
| 1.6 动手练习与验收               | 4         |
| <b>第二章 二维坐标、时间与接口</b>     | <b>5</b>  |
| 2.1 一个点可以有不同坐标            | 5         |
| 2.2 从公式到一个小函数             | 6         |
| 2.3 逆变换与变换链               | 6         |
| 2.4 运行与理解数值误差             | 6         |
| 2.5 练习: 自己写一次逆变换          | 7         |
| 2.6 仿真时间与真实等待时间           | 7         |
| 2.7 把坐标放到 ROS 的 TF 树中     | 8         |
| 2.8 暂停、单步与重置实验            | 8         |
| <b>第三章 一个可复现的二维世界</b>     | <b>10</b> |
| 3.1 先看世界, 再谈算法            | 10        |
| 3.2 圆心不碰撞, 不代表圆盘不碰撞       | 11        |
| 3.3 点到线段的距离               | 11        |
| 3.4 随机多边形为什么从凸包开始?        | 12        |
| 3.5 先检查可达, 再讲最短路径         | 12        |
| 3.6 参数、练习与验收              | 13        |
| <b>第四章 让圆盘运动: 位置、速度与力</b> | <b>14</b> |
| 4.1 圆心状态与圆盘外形             | 14        |

|                              |    |
|------------------------------|----|
| 4.2 先让机器人到一个点 . . . . .      | 14 |
| 4.3 扫过的空间也要安全 . . . . .      | 15 |
| 4.4 理想到点不产生真实 IMU . . . . .  | 16 |
| 4.5 冻结试验与重新开始 . . . . .      | 16 |
| 4.6 给速度：先分清它在哪个坐标系 . . . . . | 17 |
| 4.7 限速、超时与圆弧安全 . . . . .     | 18 |
| 4.8 接口、练习与验收 . . . . .       | 19 |

# 第一章 环境搭建与第一个圆盘场景

**本章目标** 在主机 ROS 2 上编译两个小包，启动一个节点，在 RViz2 看见半径为 0.20 m 的圆盘。

**先修** 会打开终端、编辑文本，读懂基本 C++ 变量和函数。

## 1.1 先认识四个名字

节点 (node) 是执行计算并收发消息的程序。话题 (topic) 是带名字和类型的数据通道，例如本章的 `/visualization/robot`。包 (package) 组织源码、依赖和安装规则。工作空间把多个包放到一起编译；它不是新的 ROS 发行版。

本章只有一个发布节点和一个可视化程序：



RViz2 显示收到的数据，不负责机器人动力学。本章圆盘是一个静态图形。

## 1.2 确认主机环境

本课程验证平台是 Ubuntu 26.04 和 ROS 2 Lyrical。已有主机不用重装。打开 `bash`，先运行：

```
source /opt/ros/lyrical/setup.bash
printenv ROS_DISTRO
ros2 pkg prefix rviz2
colcon version-check --help
```

前三项应分别加载环境、输出 `lyrical`、找到 `RViz2`。最后一项只是检查 `colcon` 命令，后文构建才是实际工具链验收。

干净主机应先按 [ROS 官方安装步骤](#) 配置 UTF-8、Universe 与 `ros-apt-source`。已配好软件源的 Ubuntu 26.04 可安装 `ros-lyrical-desktop`、`ros-dev-tools`、`libeigen3-dev`。不要把其他 Ubuntu 版本与 Lyrical 的二进制包混用。

Lyrical 的 ROS 接口按 C++20 编译；本课程独立几何和数学代码采用 C++17。读者不必先学习 `concepts` 等高级特性。ROS 的 Python 包使用系统 Python，构建时显式指定 `/usr/bin/python3`。

## 1.3 构建并启动

以下路径是当前主机仓库位置；换机器后改成自己的路径。每个新终端都要先加载主机环境，再加载课程工作空间。

```
source /opt/ros/lyrical/setup.bash
cd /home/zdp/ForCodex/ros2_motion_planning_course/ros2_ws
colcon build --symlink-install --cmake-args \
  -DCMAKE_BUILD_TYPE=RelWithDebInfo \
  -DPython3_EXECUTABLE=/usr/bin/python3
source install/setup.bash
ros2 launch motion2d_bringup ch01.launch.py
```

motion2d 装算法和节点；motion2d\_bringup 装配置、launch 与 RViz2 视图。--symlink-install 方便开发时编辑配置。参数只在启动时读取，所以修改 YAML 后重启节点。

RViz2 应显示浅色背景、0.5 m 网格和原点的青色圆盘，Fixed Frame 为 map。无图形界面可加 rviz:=false；这不能替代图形验收。

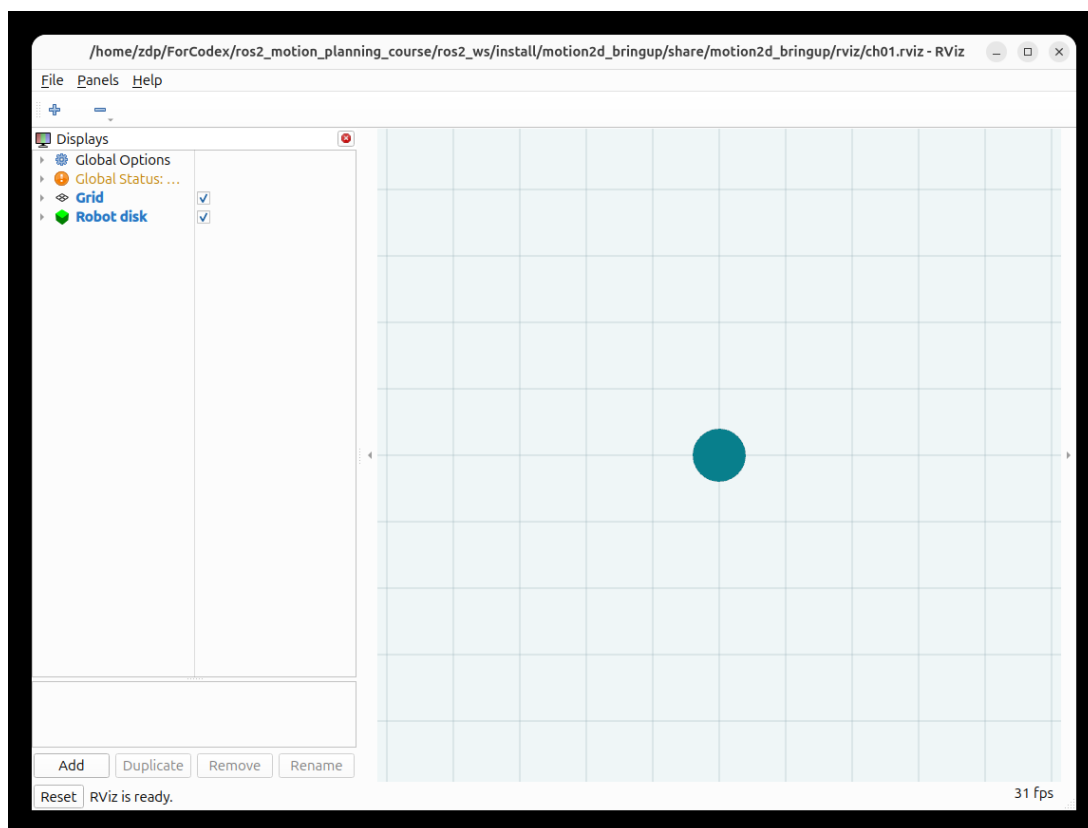


图 1.1: 主机实际运行画面。圆盘直径为 0.40 m，小于一个 0.5 m 网格。

此时 Global Status 可能提示尚无 TF 数据，因为本章还没有发布坐标树。Marker 已直接在 map 帧中，所以仍可正确显示；下一章建立 TF 后该提示消失。

## 1.4 圆盘为什么用直径?

用一个很薄的 CYLINDER 表示圆盘。Marker 的 scale 是物体的完整尺寸，因此

$$d_x = d_y = 2r, \quad r = 0.20 \text{ m} \Rightarrow d_x = d_y = 0.40 \text{ m}. \quad (1.1)$$

scale.z=0.05 只是显示厚度，后续平面碰撞只使用半径。单位四元数的 w 分量为 1；alpha 为 0 的图形完全透明。

下面直接读取实际源码的标记区域。修改代码后重新编译本书，摘录会同步更新：

```

33 visualization_msgs::msg::Marker disk;
34 disk.header.frame_id = "map";
35 disk.header.stamp = now();
36 disk.ns = "robot";
37 disk.id = 0;
38 disk.type = visualization_msgs::msg::Marker::CYLINDER;
39 disk.action = visualization_msgs::msg::Marker::ADD;
40 disk.pose.position.x = x;
41 disk.pose.position.y = y;
42 disk.pose.position.z = 0.025;
43 disk.pose.orientation.w = 1.0;
44 disk.scale.x = 2.0 * radius;
45 disk.scale.y = 2.0 * radius;
46 disk.scale.z = 0.05;
47 disk.color.r = 0.03F;
48 disk.color.g = 0.50F;
49 disk.color.b = 0.55F;
50 disk.color.a = 1.0F;
51 //

```

## 1.5 发布一次，为什么晚启动也能看到?

本节点使用 reliable + transient-local QoS，保留最后一条 Marker。RViz2 订阅同样配置持久性，晚启动时可取到保留的消息。节点退出后，不应假设消息仍会被保留。

第二个终端加载环境，进入工作空间后运行：

```

ros2 topic echo /visualization/robot --once \
--qos-durability transient_local
/usr/bin/python3 ../scripts/check_ch01.py

```

检查程序确实作为晚加入的订阅者接收消息，并核对帧、类型与默认尺寸。此时消息 stamp 用系统时间；下一章再引入仿真时钟。

## 1.6 动手练习与验收

1. **理解：**节点与话题的区别是什么？发布者先启动、RViz2 后启动时，为什么仍能收到圆盘？
2. **代码：**完成 `exercises/ch01/starter/marker_scale.hpp` 中的 `EXERCISE(ch01-1)`。检查方法、分级提示和独立参考答案位于同目录。
3. **实验：**在 `ch01.yaml` 中把半径改成 0.35 m、x 改为 1.0 m，重启。先预测消息和网格上的尺寸，再观察；检查默认配置前恢复参数。

应看到直径 0.70 m、圆心 (1,0) 的圆盘。若图形不见了，依次检查 Fixed Frame、话题、alpha 与 QoS。若找不到包，检查是否加载课程的 install 环境。

### 独立编译练习

练习的输入是半径，输出是 Marker 的 x/y 尺寸。下面的 starter 故意保留一个需要你修改的返回值；它没有被链接到正式演示中。

```
#pragma once

/** @brief Return Marker x/y size (m) for a disk of positive radius (m). */
inline double markerDiameter(double radius)
{
    // EXERCISE(ch01-1): RViz's cylinder scale describes its full width.
    return radius;
}
```

从仓库根目录运行：

```
g++ -std=c++17 -Wall -Wextra -Iexercises/ch01/starter \
    exercises/ch01/check.cpp -o /tmp/ch01_exercise
/tmp/ch01_exercise
```

未完成时检查输出 FAIL；改对后输出 PASS。检查包含两个半径，避免只为一个例子填写常数。若卡住，先看同目录的 hints，再读 solutions 的解释；把编译命令中的 starter 换成 solutions 即可验证参考解。

本章完成后，你能解释图形由哪个节点发布、消息中哪些量决定尺寸，以及如何让另一个程序接收这条消息。

## 第二章 二维坐标、时间与接口

**目标** 正确旋转、平移、求逆，并用一个仿真时钟驱动坐标树。

**先修** 第 1 章；二维向量和正弦、余弦。矩阵只需要理解本章的两行乘法。

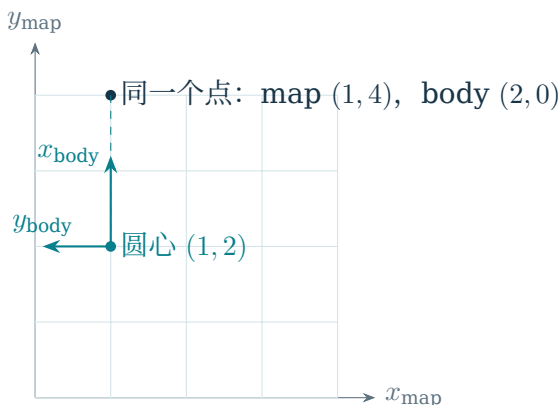
### 2.1 一个点可以有不同坐标

机器人位于地图坐标 (1, 2)，朝向正北。雷达观测到正前方 2 m 的点。它在机体系是 (2, 0)，在地图系是 (1, 4)。坐标变了，物理上的点没有变。

我们采用右手系：机体 x 轴向前、y 轴向左、z 轴向上，逆时针偏航为正，角度使用弧度。一个平面位姿记作  $T_{A \leftarrow B} = (t, \psi)$ ：它把 B 系的坐标换成 A 系坐标。

$$R(\psi) = \begin{bmatrix} \cos \psi & -\sin \psi \\ \sin \psi & \cos \psi \end{bmatrix}, \quad p_A = R(\psi)p_B + t. \quad (2.1)$$

先旋转向量，再加上原点位置。不能先加  $t$  再旋转，那样会把平移也绕 A 系原点转动。



## 2.2 从公式到一个小函数

Pose2D 只保存一个二维向量和一个角度。几何库不包含 ROS 头文件，可以单独用 C++17 编译、调试和测试。

```

13 Eigen::Matrix2d rotation(double yaw)
14 {
15     const double c = std::cos(yaw);
16     const double s = std::sin(yaw);
17     Eigen::Matrix2d result;
18     result << c, -s, s, c;
19     return result;
20 }
21
22 Eigen::Vector2d transformPoint(const Pose2D & pose, const Eigen::Vector2d & point)
23 {
24     return rotation(pose.yaw) * point + pose.position;
25 }
26 //
```

## 2.3 逆变换与变换链

从式 (2.1) 两侧减去  $t$ ，再左乘  $R^\top$ ：

$$p_B = R^\top(p_A - t) = R^\top p_A - R^\top t. \quad (2.2)$$

因此逆变换的平移是  $-R^\top t$ 。旋转矩阵的逆为转置，这是二维旋转保持长度和夹角的结果。

若雷达相对机体有固定外参  $T_{B \leftarrow L}$ ，机体相对地图为  $T_{M \leftarrow B}$ ，则

$$T_{M \leftarrow L} = T_{M \leftarrow B} T_{B \leftarrow L}. \quad (2.3)$$

从右向左应用：先把雷达点放到机体系，再放到地图系。课程默认传感器位于圆心，外参为单位变换；保留显式外参能让后续实验改变安装位置。

## 2.4 运行与理解数值误差

按第 1 章的方法重新构建、加载环境，在工作空间运行：

```

ros2 run motion2d se2_demo
colcon test --packages-select motion2d \
  --event-handlers console_direct+
colcon test-result --verbose
```

示例输出机体点 (2, 0)、地图点 (1, 4)，再恢复到 (2, 0)。浮点运算中的  $\cos(\pi/2)$  可能是接近零的小数，所以比较使用绝对误差容限  $10^{-12}$ ，不是要求每个计算结果逐位相同。

角度是另一个边界问题。179° 到 -179° 的最短旋转只有 2°。先将差值归一到  $[-\pi, \pi)$ ，再把它作为误差。函数 `wrapAngle` 使用弧度；输入 90 表示 90 rad，不是 90 度。

## 2.5 练习：自己写一次逆变换

1. **推导：**为何逆变换中不能只把  $t$  取负？用  $\psi = 90^\circ, t = (1, 2)$  给一个反例。
2. **代码：**补完 `exercises/ch02/starter/inverse_point.hpp` 的标记函数。它复用正式库的 `Pose2D`，检查包括两个旋转方向。
3. **实验：**把示例朝向换成  $-90^\circ$ ，预测地图点，再运行核对。还可以将激光外参设为机体前方 0.5 m，手算其地图位置。

独立编译命令、分级提示和参考答案位于同章习题 README。数学库的自动测试另外覆盖手算变换、传感器偏移、200 次固定种子的往返和角度边界。

## 2.6 仿真时间与真实等待时间

假设每个仿真步长是  $\Delta t = 0.005$  s。第  $k$  步的仿真时间为

$$t_k = k\Delta t. \quad (2.4)$$

即使电脑偶尔调度得慢，状态仍然只前进一个固定步长。运行速度可能变慢，但数学实验的步长没有偷偷改变。实时性能是另一项需要测量的性质。

`SimClock` 用整数 `tick` 和整数纳秒保存时间，避免反复执行浮点加法。暂停时自动 `advance` 返回 `false`，单步只在暂停时允许执行：

```

14 bool SimClock::advance()
15 {
16     if (paused_) {
17         return false;
18     }
19     ++ticks_;
20     return true;
21 }
22
23 bool SimClock::singleStep()
24 {
25     if (!paused_) {
26         return false;
27     }
28     ++ticks_;
29     return true;
30 }
31 //
```

本章由 `frame_demo_node` 唯一发布 `/clock`；RViz2 和节点都设置 `use_sim_time=true`。暂停后仍会重复发送相同时间值，便于新订阅者获得当前时间，但状态和时间戳不再前进。不要把消息数量误认为仿真步数。

## 2.7 把坐标放到 ROS 的 TF 树中

$\text{map} \rightarrow \text{odom} \rightarrow \text{base\_link} \rightarrow \{\text{laser}, \text{imu\_link}\}.$

本章 map 到 odom 为单位变换；传感器位于圆心、与机体系对齐，也是单位变换。这三条静态边通过 /tf\_static 发布。odom 到 base\_link 是带采样时刻的动态边，通过 /tf 发布。

为观察旋转效果，示例固定圆心 (1, 2)，采用解析朝向

$$\psi(t) = \frac{\pi}{2} + 0.2t. \quad (2.5)$$

这是一套坐标演示装置，还没有受力动力学和真实激光模型。图中的蓝色箭头只是 laser 帧中长度为 2 m 的向量。

ROS 使用四元数表示旋转。平面 yaw 对应  $(q_x, q_y, q_z, q_w) = (0, 0, \sin(\psi/2), \cos(\psi/2))$ 。发布时刻直接取仿真时钟，不能换成回调到达的真实时间：

```

126 geometry_msgs::msg::TransformStamped transform;
127 transform.header.stamp = rclcpp::Time(clock_.nanoseconds(), RCL_ROS_TIME);
128 transform.header.frame_id = "odom";
129 transform.child_frame_id = "base_link";
130 transform.transform.translation.x = x_;
131 transform.transform.translation.y = y_;
132 const double angle = wrapAngle(yaw_ + yaw_rate_ * clock_.seconds());
133 transform.transform.rotation.z = std::sin(angle / 2.0);
134 transform.transform.rotation.w = std::cos(angle / 2.0);
135 broadcaster_.sendTransform(transform);
136 //

```

## 2.8 暂停、单步与重置实验

停止第 1 章程序，重新构建并加载环境后启动：

```
ros2 launch motion2d_bringup ch02.launch.py
```

另一个终端同样加载环境后调用：

```

ros2 service call /sim/pause std_srvs/srv/SetBool "{data: true}"
ros2 service call /sim/step std_srvs/srv/Trigger "{}"
ros2 service call /sim/reset std_srvs/srv/Trigger "{}"
ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"

```

暂停时圆盘朝向和箭头冻结；每次单步前进 5 ms；reset 回到时间零、初始朝向，并保持暂停。运行中请求单步会明确失败，防止把手动步数混入自动积分。

在工作空间运行 /usr/bin/python3 ../scripts/check\_ch02.py 可检查唯一 TF 父帧、解析朝向、冻结时钟、5 ms 单步、reset 和恢复运行。它会改变演示状态，仅用于本章实验。

重置会使仿真时间向后跳，TF 监听器会清空缓存，这是新试验的正常边界。本节点每隔 1 s 墙钟时间重发静态外参，暂停时同时重发当前动态边，使监听器能在 reset 后恢复。状态并未因此前进。后续 reset 还须清理估计和传感器历史。

## 时间练习

先暂停，再连续调用 `step` 十次，预测仿真时间增量和朝向增量，之后查看 `/clock` 和 TF 核对。标准答案是 0.05 s 与 0.01 rad。改变 YAML 中的 `dt` 后要重新推导；不应继续照搬默认值的检查脚本。

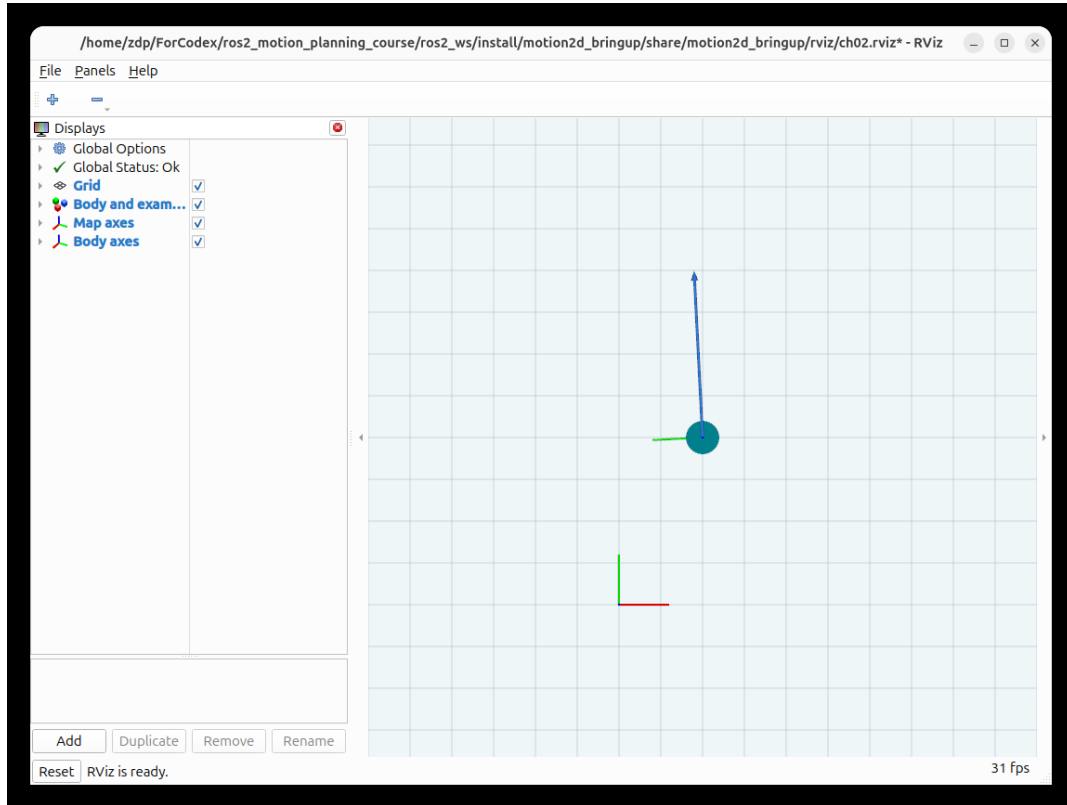


图 2.1: 重置并恢复运行后的主机 RViz2。圆盘与机体向量正确显示，TF 状态正常。

## 第三章 一个可复现的二维世界

**目标** 生成随机圆与凸多边形，显示起终点，检查有半径的机器人能否放入世界。

**先修** 前两章；向量点积。凸包与距离的必要直觉在本章说明。

### 3.1 先看世界，再谈算法

停止之前的演示，重新构建并加载工作空间后运行：

```
ros2 launch motion2d_bringup ch03.launch.py
```

默认世界宽、高都是 20 m，边界以原点为中心。 $\text{seed}=42$  时生成 8 个圆和 8 个凸多边形；机器人在  $(-8, -8)$ ，目标在  $(8, 8)$ 。世界是静态的，机器人尚未运动。

此处显示的是仿真掌握的完整几何。后续激光只能看见无遮挡且在量程内的表面，建图不能直接读取这些障碍物顶点。理解这个数据边界，才能公平比较真值模式和 SLAM 模式。

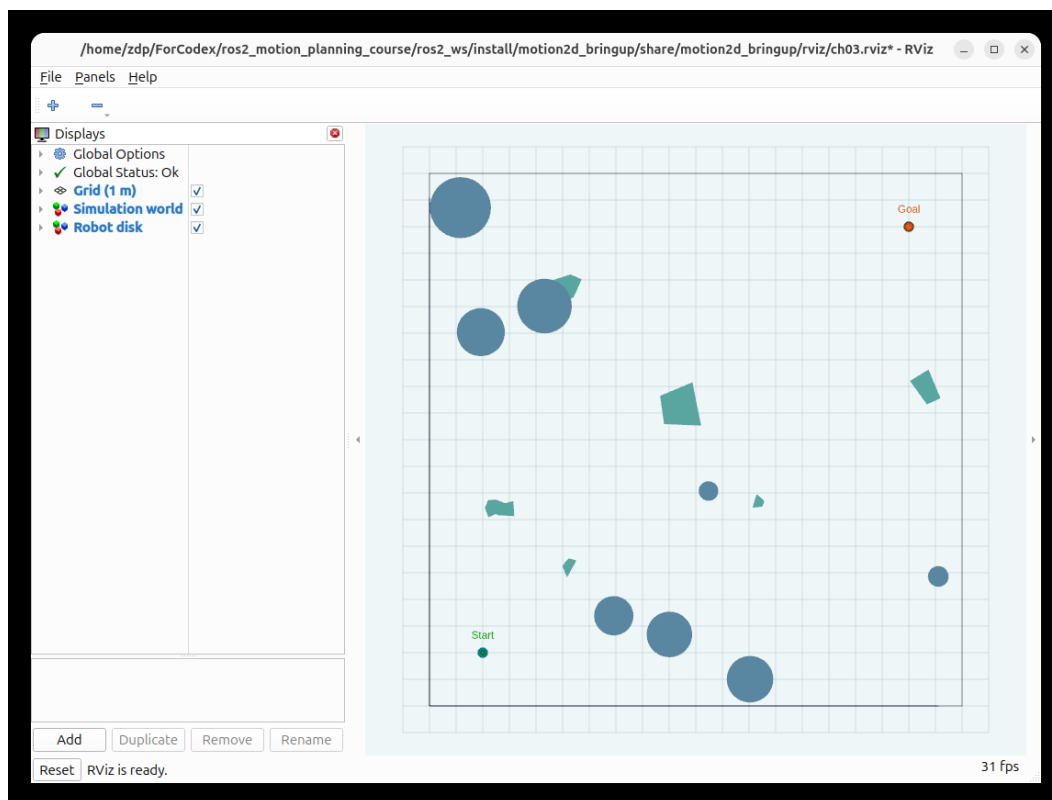


图 3.1:  $\text{seed}=42$  的实际世界。圆和凸多边形可以重叠；Start 与 Goal 分别标出起终点。

## 3.2 圆心不碰撞，不代表圆盘不碰撞

半径为  $r$  的圆盘中心位于  $p$ ，圆形障碍中心为  $c$ 、半径为  $R$ 。两者相切时，中心距为  $r + R$ ，本课程把接触计为碰撞：

$$\|p - c\| > r + R \iff \text{与该障碍无碰撞.} \quad (3.1)$$

定义单个障碍的有符号距离  $d(p)$ ：外部为正、内部为负。自由区域中，圆盘需要  $d(p) > r$ 。如要额外留出安全裕量  $\delta$ ，使用  $d(p) > r + \delta$ ；物理半径与规划裕量是两个不同参数。

世界边界也限制可用空间。核心函数在自由域中取边界和所有障碍的最小距离，再比较半径：

```

11 double clearance(const World2D & world, const Eigen::Vector2d & center)
12 {
13     double distance = std::min(
14         world.width / 2.0 - std::abs(center.x()),
15         world.height / 2.0 - std::abs(center.y()));
16     for (const auto & circle : world.circles) {
17         distance = std::min(distance, signedDistance(circle, center));
18     }
19     for (const auto & polygon : world.polygons) {
20         distance = std::min(distance, signedDistance(polygon, center));
21     }
22     return distance;
23 }
24
25 bool isFree(const World2D & world, const Eigen::Vector2d & center, double radius)
26 {
27     return clearance(world, center) > radius;
28 }
29 //

```

占据区或越界时，这个函数用于判断符号，不承诺给出并集内部的精确 ESDF；第 12 章会专门构建距离场。

## 3.3 点到线段的距离

多边形边界由线段组成。设边为  $a \rightarrow b$ ，把点  $p$  投影到直线上：

$$t^* = \frac{(p - a)^\top (b - a)}{\|b - a\|^2}, \quad t = \min(1, \max(0, t^*)), \quad q = a + t(b - a). \quad (3.2)$$

$q$  是线段上离  $p$  最近的点，距离是  $\|p - q\|$ 。限制  $t$  很重要：离线段延长线很近，不代表离有限线段很近。若  $a=b$ ，则线段退化为一个点。

```

17 double pointSegmentDistance(
18     const Eigen::Vector2d & point, const Eigen::Vector2d & a, const Eigen::Vector2d & b)
19 {
20     const Eigen::Vector2d edge = b - a;
21     if (edge.squaredNorm() == 0.0) {
22         return (point - a).norm();
23     }
24     const double t = std::clamp((point - a).dot(edge) / edge.squaredNorm(), 0.0, 1.0);
25     return (point - (a + t * edge)).norm();
26 }
27 //

```

对逆时针凸多边形，内部点位于每条有向边的左侧。若点在内部，距离取负；外部则取到全部边的最小距离。

### 3.4 随机多边形为什么从凸包开始？

随意把随机点连起来可能产生自交。我们的步骤是：在一个采样圆盘内生成点，再取凸包。可把凸包想成绕所有点拉紧的橡皮筋。实现先按  $x/y$  排序，再维护上、下两条只向同一方向拐弯的链。

若  $u, v$  独立服从  $[0, 1)$  均匀分布，使用

$$\theta = 2\pi u, \quad \rho = R\sqrt{v}, \quad p = c + \rho(\cos \theta, \sin \theta) \quad (3.3)$$

让采样点在圆盘面积上均匀。因为半径不超过  $\rho$  的面积比例是  $\rho^2/R^2$ ，所以半径要取平方根。

代码中 `polygon_samples=8` 表示采样八个点，凸包顶点通常更少。校验要求顶点有限、严格凸、逆时针；对每条边检查其余顶点都位于左侧，也能排除自交星形。

不同障碍可以重叠，这仍是合法障碍并集。世界生成器只拒绝无效形状、越界和侵入起终点保护圆的候选。每个形状最多尝试 1000 次，失败明确报告，不偷偷换种子。

### 3.5 先检查可达，再讲最短路径

起点、终点各自无碰撞，并不意味着中间一定连通。我们先用粗网格和四邻接 `flood fill` 检查连通性：从起点所在格入队，反复访问相邻自由格，能到终点格即通过。它不寻找最短路径，也不替代第 11 章的  $A^*$ 。

不能仅检查格中心。对于宽  $\Delta x$ 、高  $\Delta y$  的单元，中心到最远角点的距离为

$$\epsilon_{\text{cell}} = \frac{1}{2} \sqrt{\Delta x^2 + \Delta y^2}. \quad (3.4)$$

中心只有在净空大于  $r + \delta + \epsilon_{\text{cell}}$  时才标为自由。这保证整个单元能容纳圆盘，代价是窄通道可能被保守拒绝。

默认检查分辨率 0.25 m。若失败，可以明确减小分辨率或关闭 `require_connected` 观察场景；同一个 `seed` 不会自动被替换成“更好看”的世界。记录失败案例也是学习的一部分。

## 3.6 参数、练习与验收

编辑 `ros2_ws/src/motion2d_bringup/config/ch03.yaml` 后重启。公共 `x/y/radius` 同时提供给坐标节点和世界节点；圆盘显示与碰撞不应使用两套不同半径。

1. **理解：**给一个圆心在自由空间、圆盘却碰撞的例子。解释相切和安全裕量。
2. **代码：**实现 `exercises/ch03/starter/segment_distance.hpp` 的点线段距离。独立检查覆盖内点、端点外、线段上和退化线段；提示与参考解单独存放。
3. **实验：**相同配置和 `seed=42` 启动两次，比较地图。然后改为 43，再把两类障碍各增至 20 个，记录可达性和起终点净空。

代码练习从仓库根目录单独编译，不依赖 ROS：

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iexercises/ch03/starter \
exercises/ch03/check.cpp -o /tmp/ch03_exercise
/tmp/ch03_exercise
```

将 `starter` 替换为 `solutions` 可以检验参考解。先运行未完成的练习观察失败位置，再修改投影公式；不要通过删掉检查来得到“通过”。

在同一标准库和参数下，固定种子复现相同世界；跨编译器标准库版本不保证分布函数的逐位一致。输出状态说明是否执行了连通性检查，避免把“没有检查”当成“已经通过”。

运行 `colcon test` 检查几何、碰撞和可达性；演示运行时，在工作空间执行以下命令核对默认消息：

```
/usr/bin/python3 ../scripts/check_ch03.py
```

RViz2 展示全部障碍、边界和起终点，图形并非激光建图结果。

### 常见问题

看不见障碍时，先确认 Fixed Frame 为 `map`、World 的话题为 `/visualization/world`，再检查节点有没有因无效参数退出。改参数后要重启；本章尚不提供运行中重建世界。若可达性检查失败，保留 `seed` 和配置，再研究分辨率与通道宽度。

**主机观察：**本机 RViz2 曾在一次较长演示的关闭阶段崩溃，运行显示正常，后续复测未再出现，原因尚未确定。启动文件已避免重复关闭事件；若需要分开观察进程，用 `rviz:=false` 启动节点，再按本章 README 的命令独立打开 RViz2，先关闭 RViz2，再停止节点。

## 第四章 让圆盘运动：位置、速度与力

**目标** 区分理想位置执行与物理运动；让碰撞半径真正约束移动。

**先修** 前三章、速度与加速度的含义。先运行，再推导。

### 4.1 圆心状态与圆盘外形

机器人用圆心描述状态，圆盘描述占据的空间：

$$x = (p, v, \psi, \omega), \quad p = (p_x, p_y), \quad v = (v_x, v_y). \quad (4.1)$$

位置单位 m，速度 m/s，yaw 为 rad，角速度 rad/s。位置和速度在 odom 系表达；本章的 map 与 odom 重合。半径 r 单独参与碰撞，不把“质点状态”误解为“没有体积”。若  $r=0$ ，才是几何质点；屏幕上的小点仅用于看清位置。

控制量不同，模型的物理意义就不同。理想到点输入目标位姿；速度模型输入平面速度；惯性模型输入力和偏航力矩。它们分别使用不同话题，不能把同一个数值悄悄解释成另一种单位。先完成最容易运行的理想到点，再逐项加入连续运动模型。

### 4.2 先让机器人到一个点

停止之前的场景，重新构建并加载 install 后运行：

```
ros2 launch motion2d_bringup ch04.launch.py model:=ideal
```

在 RViz2 用 2D Goal Pose 工具点击、拖动，指定目标位置和 yaw。工具向 /command/pose 发布消息。也可在第二个已加载 ROS 环境的终端发送：

```
ros2 topic pub --once /command/pose geometry_msgs/msg/PoseStamped \
  "{header: {frame_id: odom}, pose: {position: {x: -7.0, y: -8.0}, orientation: {w: 1.0}}}"
```

默认从  $(-8, -8)$  移到  $(-7, -8)$ ，图中留下接受过的运动轨迹。目标在下一 tick 才执行；暂停时发目标，圆盘保持不动，单步后才到达目标。

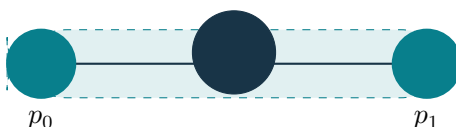
## 4.3 扫过的空间也要安全

起终点安全并不足够。半径  $r$  的圆盘沿线段  $[p_0, p_1]$  移动，扫过的区域是胶囊形。对于圆心  $c$ 、半径  $R$  的障碍，无碰撞要求

$$\text{dist}(c, [p_0, p_1]) > R + r. \quad (4.2)$$

这里直接复用第 03 章的点线段距离。对凸多边形，检查起终点和运动线段到每条边的距离；对矩形边界，内缩后的矩形是凸集，检查两个端点即可保证整条直线留在其中。

端点自由，整段被挡住



```

32 bool sweptDiskIsFree(const World2D & world, const Eigen::Vector2d & from,
33   const Eigen::Vector2d & to, double radius)
34 {
35   // The inset rectangle is convex, so valid endpoints imply a valid segment.
36   if (!isFree(world, from, radius) || !isFree(world, to, radius)) {return false;}
37   for (const auto & circle : world.circles) {
38     if (pointSegmentDistance(circle.center, from, to) <= radius + circle.radius) {
39       return false;
40     }
41   }
42   for (const auto & polygon : world.polygons) {
43     const auto & v = polygon.vertices;
44     for (std::size_t i = 0; i < v.size(); ++i) {
45       if (segmentDistance(from, to, v[i], v[(i + 1) % v.size()]) <= radius) {
46         return false;
47       }
48     }
49   }
50   return true;
51 }
52 //

```

代码没有按固定间距抽样，因此不会因为障碍很薄、目标很远，就从障碍另一侧“穿过去”。这是连续直线扫掠的几何检查，浮点实现仍受数值精度限制。

## 4.4 理想到点不产生真实 IMU

通过几何检查后，理想模型直接赋值目标位姿：

```
6 State2D idealPose(const Pose2D & target)
7 {
8     State2D result;
9     result.pose = target;
10    result.pose.yaw = wrapAngle(target.yaw);
11    return result; // Zero derivatives mean "unused", not an IMU measurement.
12 }
13 //
```

位置在一个 tick 中改变，不代表机器人以一个有限速度完成了运动。速度和加速度字段为未使用的零值；不能对这种跳变做差分，再把结果称为模拟 IMU。平滑轨迹执行将在轨迹章节加入。

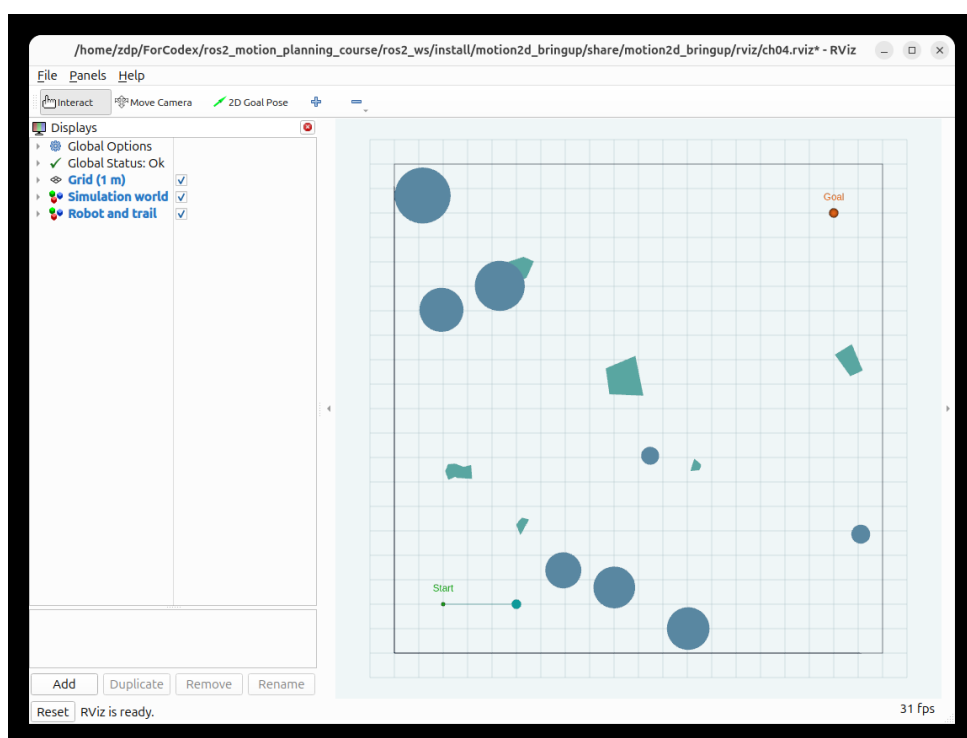


图 4.1: 实际 RViz2 画面：世界、圆盘与已接受的移动轨迹。

## 4.5 冻结试验与重新开始

若扫描检查失败，程序不接受候选状态：位置、速度和时间停在上一个通过检查的 tick，/sim/status 为 collision\_predicted，圆盘变红。这是提前终止试验，尚未模拟接触力、反弹或摩擦。它没有把撞入障碍的状态推回自由空间。需要调用 reset 才能继续。

```
ros2 service call /sim/reset std_srvs/srv/Trigger "{}"
ros2 service call /sim/step std_srvs/srv/Trigger "{}"
ros2 service call /sim/pause std_srvs/srv/SetBool "{data: false}"
```

reset 开始新试验：回初始状态、清空待执行命令与轨迹、仿真时间归零并暂停。单步推进一个 dt；恢复运行不改变 dt。墙钟定时器决定运行快慢，整数 tick 决定模拟时间。

## 4.6 给速度：先分清它在哪个坐标系

重启场景，选用速度模型：

```
ros2 launch motion2d_bringup ch04.launch.py model:=velocity
```

此时只订阅 `/command/velocity`，RViz2 的位姿目标不参与驱动。在另一个已加载环境的终端连续发送机体系速度：

```
ros2 topic pub -r 20 /command/velocity geometry_msgs/msg/TwistStamped \
  "{header: {frame_id: base_link}, twist: {linear: {x: 0.5}, angular: {z: 0.3}}}"
```

`odom` 系速度方向固定；`base_link` 系速度会随机器人转动。同一个  $(0.5, 0)$ ，前者表示一直向 `odom` 的  $x$  正方向走，后者表示一直朝圆盘的“前方”走。全向模型也允许  $y$  方向输入，这和不能横移的差速车不同。

若一个时间步  $h$  内保持 `odom` 速度  $u$  与角速度  $\omega$ ，则

$$p_{k+1} = p_k + hu, \quad \psi_{k+1} = \text{wrap}(\psi_k + h\omega). \quad (4.3)$$

若  $u$  在机体系，积分  $\dot{p} = R(\psi_k + \omega t)u$  可得

$$p_{k+1} = p_k + h \text{sinc}(\beta) R(\psi_k + \beta)u, \quad \beta = \frac{\omega h}{2}. \quad (4.4)$$

其中  $\text{sinc}(\beta) = \sin \beta / \beta$ ，在零处取 1。小角度用  $1 - \beta^2/6$  避免除以很小的数。这个式子覆盖圆弧与直线极限：

```
16 State2D stepVelocity(const State2D & state, const VelocityCommand & command, double dt)
17 {
18     State2D next = state;
19     const double half_turn = 0.5 * command.yaw_rate * dt;
20     next.pose.yaw = wrapAngle(state.pose.yaw + 2.0 * half_turn);
21     next.yaw_rate = command.yaw_rate;
22     next.yaw_acceleration = 0.0; // Only within the held-command interval.
23     next.acceleration.setZero();
24     if (command.body_frame) {
25         const double sinc = std::abs(half_turn) < 1e-6 ?
26             1.0 - half_turn * half_turn / 6.0 : std::sin(half_turn) / half_turn;
27         next.pose.position += dt * sinc * rotation(state.pose.yaw + half_turn) * command.velocity;
28         next.velocity = rotation(next.pose.yaw) * command.velocity;
29         next.acceleration = command.yaw_rate * Eigen::Vector2d{-next.velocity.y(), next.velocity.x()};
30     } else {
31         next.pose.position += dt * command.velocity;
32         next.velocity = command.velocity;
33     }
34     return next;
35 }
36 //
```

## 4.7 限速、超时与圆弧安全

默认平移速度模长限制为 1 m/s, 角速度绝对值限制为 1 rad/s。超出平移限制时等比例缩短向量, 保留方向; 不能分别限每个分量后, 仍声称限制了模长。配置项是 `speed_max` 和 `yaw_rate_max`。

命令默认在模拟时间中有效 0.5 s, 到期后在 `tick` 边界切为零速度, 状态为 `command_timeout`。暂停不会消耗有效期。零时间戳从接收时刻起算, 非零时间戳从消息时刻起算。改变 `dt` 时, 过期检测最多量化到一个 `tick`。由于可以瞬间改变速度, 该模型依然不用于 IMU 融合; 后续惯性模型需要真实制动。

圆弧的中间部分不一定贴着起终点连线。设加速度模长在一个步长内不超过  $a_{\max}$ , 曲线与线性插值的偏差满足

$$\|p(t) - p_{\text{chord}}(t)\| \leq \frac{a_{\max}}{2}t(h-t) \leq \frac{a_{\max}h^2}{8}. \quad (4.5)$$

可把它理解为线性插值的二阶误差界。机体系恒速度转弯时  $a_{\max} = |\omega| \|u\|$ , `odom` 恒速度时为零。因此把检查半径暂时改为  $r + a_{\max}h^2/8$ , 再复用直线扫掠检查, 就把整段曲线包在胶囊内。这是保守的充分检查, 可能拒绝狭窄但实际上可通过的路径。

**为什么不能只检查两个状态?** 薄墙可能位于两个 `tick` 之间; 转弯时障碍也可能位于圆弧上、却不在端点连线上。测试专门构造了后一种场景, 验证“只检查弦”会漏检, 加上曲率引起的余量后能拒绝。

**代码练习 ch04-2:** 补完 `exercises/ch04/starter/velocity_step.hpp` 的 `odom` 系速度积分, 并归一化 `yaw`。用 `check_velocity.cpp` 检查负速度、时间步长和跨越 `pi` 的角度。编译时沿用本章习题 README 的命令。

**实验:** 在转向后分别发送 `odom` 与 `base_link` 的同一速度, 比较轨迹; 停止发送, 验证超时; 暂停两秒再单步, 解释为什么暂停不消耗模拟时间。运行下面的实际通信检查:

```
/usr/bin/python3 ../scripts/check_ch04.py --model velocity
```

## 4.8 接口、练习与验收

本章默认  $dt=0.005$  s。位姿输入接受 `odom/map`；`map` 可用是因为本章二者重合。姿态必须是平面单位四元数，不能把四个分量全填零。时间戳为零表示下一 `tick`；非零时间戳只接受当前模拟时刻附近的命令，过旧或过远的未来命令被拒绝。

`/sim/pose`、`/sim/velocity`、`/sim/acceleration` 都是真值调试输出，平移分量在 `odom` 系。第 07 章才引入规范的里程计消息与选源，后续 SLAM 不得读取这些调试话题。本章没有激光或 IMU。

1. **理解：** 给一个起终点都自由但中间碰撞的例子。为什么瞬时位置赋值不适合 IMU？
2. **代码 ch04-1：** 完成 `exercises/ch04/starter/sweep_circle.hpp`。复用点线段距离，判定圆盘与圆障碍的整段扫掠；相切必须判为碰撞。
3. **实验：** 暂停后发目标并单步；改变半径重复同一条贴近障碍的移动。记录 `seed`、半径、起终点和结果。不要把显示尺寸当作碰撞半径。

从仓库根目录编译练习：

```
g++ -std=c++17 -I/usr/include/eigen3 \
-Iros2_ws/src/motion2d/include -Iexercises/ch04/starter \
exercises/ch04/check_sweep.cpp \
ros2_ws/src/motion2d/src/geometry/obstacles.cpp \
-o /tmp/ch04_sweep
/tmp/ch04_sweep
```

提示和参考解分开存放；将 `starter` 替换为 `solutions` 可以验证参考解。演示运行时，从工作空间执行：

```
/usr/bin/python3 ../scripts/check_ch04.py
```

脚本会改变场景状态，检查暂停命令、单步、错误帧、碰撞冻结与 `reset`。算法测试另外覆盖薄墙和零半径质点。

**没有移动时先检查** 是否暂停、是否需要 `reset`、模型是否与输入话题对应，再检查 `frame` 和四元数。只运行一个课程 `launch`，避免多个时钟或 TF 发布者。